Clone

Distil a 70B into a 3B. Run it free and offline. Build cost: ~$12.

@keshavsuki -- Together AI + distilabel + Unsloth + Ollama

> Two corrections from the script. (1) Together AI valuation: the script says '$3.3B' -- that was their February 2025 Series B. By May 2026 they are reported to be raising $1B at a $7.5B valuation. (2) distilabel star count: '3.2K' is unconfirmed from public sources -- the GitHub repo exists (argilla-io/distilabel) but the exact count was not independently verified for this guide.

The Four Tools

Tool	Role	Stars / funding	Language / notes
`Together AI`	70B teacher model API	Seeking $7.5B val	OpenAI-compatible REST
`distilabel`	Synthetic data pipeline	~3K (unconfirmed)	Python -- argilla-io/distilabel
`Unsloth`	Fine-tune 2x faster	66K+	Python + CUDA -- exports GGUF
`Ollama`	Local inference	173K+	Runs GGUF on CPU/GPU/Metal

What Distillation Is

A small model is dumb on its own. But it can learn from a big one. You use the big model to generate thousands of perfect labeled examples for your specific task. Then you train the small model on those examples. The small model learns to replicate the big model's behavior on that narrow task -- without needing the big model's full weights.

> The result: a 3B model that performs like a 70B on one specific task. It runs locally on a MacBook at 60 tokens per second with no internet, no API key, and nothing leaving the machine.
> **For health, legal, and finance clients: the data never leaves the building. That is the unlock.**

Cost Breakdown

Together AI charges per million tokens for inference. For 1,000 training examples with judge filtering:

Item	Tokens	Cost at $0.88/1M (Llama 3.1 70B)
Generate 3,000 examples (70B teacher)	~5M tokens	~$4.40
Judge / filter (70B as scorer)	~3M tokens	~$2.64
Cloud GPU for fine-tune (A100 ~90 min)	--	~$4.50
Total		**~$11.50 -- rounds to $12**

Step 1 -- Generate Training Data with distilabel

Step 1 Build the distilabel pipeline

distilabel validates the pipeline DAG before any inference runs -- it checks input/output column declarations and connectivity. This is the free validation step. The script calls this a 'dry run.' Run it before spending anything on API calls.

```
pip install distilabel[together]

from distilabel.pipeline import Pipeline
from distilabel.steps import LoadDataFromHub, KeepColumns
from distilabel.steps.tasks import TextGeneration, UltraFeedback
from distilabel.llms import TogetherLLM

teacher = TogetherLLM(
    model='meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo',
    api_key=os.environ['TOGETHER_API_KEY'],
)

with Pipeline(name='medical-note-extractor') as pipeline:
    load = LoadDataFromHub(
        repo_id='your-seed-dataset',  # 100-200 raw medical notes
        split='train',
        batch_size=32,
    )

    generate = TextGeneration(
        llm=teacher,
        system_prompt=(
            'You are a medical data extraction assistant. '
            'Extract structured JSON from the medical note. '
            'Return only valid JSON with keys: '
            'diagnosis, medications, vitals, follow_up.'
        ),
        num_generations=3,  # generate 3 responses per note
    )

    judge = UltraFeedback(llm=teacher)  # same model as judge

    keep = KeepColumns(columns=['instruction','generation','rating'])

    load >> generate >> judge >> keep
```

```
# Validate the pipeline DAG (free -- no API calls yet)
pipeline.dry_run(parameters={
    'generate': {'llm': {'generation_kwargs': {'max_new_tokens': 512}}},
}, num_batches=1)  # run 1 batch only to confirm it works

# Full run once dry run passes
distiset = pipeline.run(parameters={
    'generate': {'llm': {'generation_kwargs': {'max_new_tokens': 512}}},
})
distiset.push_to_hub('your-org/medical-extraction-dataset')
```

> Dry run first -- always. The pipeline validates DAG structure and column declarations before touching the API. Errors here cost nothing. Errors after are billable.
> EOS token between examples -- critical. When you format your training data, every example must end with the model's end-of-sequence token (e.g. <|eot_id|> for Llama). Without it, the model cannot distinguish where one example ends and the next begins. It leaks context between examples and hallucinates on new inputs at inference time. Unsloth's SFTTrainer handles this automatically when you pass the correct chat template -- but verify it is applied before you start training.

Step 2 -- Fine-Tune with Unsloth

Step 2 Fine-tune a 3B model on your generated examples

Unsloth fine-tunes 2x faster than standard Hugging Face training with 60-70% less VRAM. Run this on a single A100 on Google Colab or RunPod. Expected: ~90 minutes for 3,000 examples on a 3B model.

```
pip install unsloth

from unsloth import FastLanguageModel
from trl import SFTTrainer
from transformers import TrainingArguments
from datasets import load_dataset

# Load base 3B model with 4-bit quantization
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name='meta-llama/Llama-3.2-3B-Instruct',
    max_seq_length=2048,
    load_in_4bit=True,
)

# Add LoRA adapters (only these layers are trained)
model = FastLanguageModel.get_peft_model(
    model,
    r=16,                     # LoRA rank
    target_modules=['q_proj','k_proj','v_proj','o_proj'],
    lora_alpha=16,
    lora_dropout=0,
)

# Load your distilabel dataset
dataset = load_dataset('your-org/medical-extraction-dataset')
```

```
trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset['train'],
    dataset_text_field='text',
    max_seq_length=2048,
    args=TrainingArguments(
        per_device_train_batch_size=4,
        gradient_accumulation_steps=4,
        num_train_epochs=3,
        learning_rate=2e-4,
        output_dir='./medical-extractor',
    ),
)
trainer.train()

# Export as GGUF for Ollama
model.save_pretrained_gguf(
    'medical-extractor-gguf',
    tokenizer,
    quantization_method='q4_k_m',  # standard 4-bit quant
)
```

Step 3 -- Deploy Locally with Ollama

Step 3 Drop the GGUF into Ollama and run

Create a Modelfile, import the GGUF, and the model is available as a local API endpoint. No internet required after this point.

```
# Create a Modelfile
cat > Modelfile << 'EOF'
FROM ./medical-extractor-gguf/unsloth.Q4_K_M.gguf
PARAMETER temperature 0.1
PARAMETER num_ctx 2048
SYSTEM You are a medical data extraction assistant. Extract structured JSON from
medical notes. Return only valid JSON.
EOF

# Build and run
ollama create medical-extractor -f Modelfile
ollama run medical-extractor

# Or call via API (same endpoint as OpenAI)
curl http://localhost:11434/api/generate -d '{
  "model": "medical-extractor",
  "prompt": "Patient presents with...[note text]",
  "stream": false
}'

# Performance on Apple Silicon MacBook (M-series):
```

```
# 3B Q4_K_M: ~60 tokens/second
# Sub-second latency for typical medical note extraction
# Zero network calls after model is loaded
```

> The model ID in the Modelfile must match the exact GGUF filename Unsloth writes. Check the output directory after save_pretrained_gguf -- the filename pattern is unsloth.Q4_K_M.gguf. If it differs, update the FROM line in the Modelfile.

TUI -- Build the Full Pipeline

Paste this into Claude Code. It scaffolds all three scripts, confirms your API key is set, and runs the distilabel dry run before spending anything.

```
Build my distillation pipeline. Create all files. Ask me for keys first.

Step 1 -- Ask me for:
  TOGETHER_API_KEY (from api.together.xyz)
  HF_TOKEN (from huggingface.co/settings/tokens -- for dataset push)
  My task description in one sentence
  My seed dataset HuggingFace repo ID (or 'none' to create one)

Step 2 -- Install dependencies:
  pip install distilabel[together] unsloth datasets trl

Step 3 -- Create pipeline.py
  [paste the full distilabel pipeline from this guide]
  Replace 'medical note extraction' with the task I gave you.
  Set TOGETHER_API_KEY from my input.

Step 4 -- Run the dry run (free validation):
  python pipeline.py --dry-run
  Show me the output. Confirm the pipeline DAG validates.
  Do NOT run the full pipeline yet.

Step 5 -- Create train.py
  [paste the full Unsloth training script from this guide]
  Set max_seq_length=2048, num_train_epochs=3

Step 6 -- Create deploy.sh
  #!/bin/bash
  ollama create $1 -f Modelfile
  echo "Model ready. Run: ollama run $1"

Step 7 -- Create Modelfile template:
  FROM ./[model-name]-gguf/unsloth.Q4_K_M.gguf
  PARAMETER temperature 0.1
  SYSTEM [task-appropriate system prompt]

When all files are created: print the full pipeline summary.
Show me what each file does and in what order to run them.
Ask me: 'Dry run passed. Ready to start the full data generation?'
```

> Do not run the full distilabel pipeline until the dry run passes. The dry run validates the pipeline DAG for free. A misconfigured pipeline that runs 3,000 examples costs ~$7 and produces unusable data. Validate first.

> **A big model teaches. A small model serves. Together AI generates the examples. distilabel judges them. Unsloth trains the 3B. Ollama runs it offline. Build cost: ~$12. Running cost: $0.**

@keshavsuki